

Day 16 — Bash + Automation

Variables, Scripts, cron & Running Python from Bash — Full Detail

Why this matters: a bash script is what actually schedules and runs your Python pipelines on a real server - this is the glue between "code that works" and "code that runs automatically every day."

1. Variables

Bash variables store text or numbers, similar to Python - but with different, stricter syntax.

```
#!/bin/bash

DATA_DIR="/home/akash/data"
FILE_NAME="sales.csv"

echo $DATA_DIR
echo "Processing file: $FILE_NAME"

# Combining variables
FULL_PATH="$DATA_DIR/$FILE_NAME"
echo $FULL_PATH
```

Rules that trip people up:

- **No spaces around =** - `DATA_DIR = "/path"` is an error; must be `DATA_DIR="/path"`
- **Use \$ to read a variable**, but not when first setting it
- **Quote variables containing paths** to avoid issues if they contain spaces

Key takeaway: Store repeated values (folder paths, filenames) in variables at the top of a script - the same "config separate from logic" instinct from Day 3, applied to bash.

2. Arguments

A bash script can accept input values when it's run, called arguments - referenced as \$1, \$2, etc.

```
#!/bin/bash
# saved as run_pipeline.sh

ENVIRONMENT=$1
DATA_FILE=$2

echo "Running in environment: $ENVIRONMENT"
echo "Processing file: $DATA_FILE"
```

```
# Running it from the terminal:
./run_pipeline.sh production sales.csv

# Output:
# Running in environment: production
# Processing file: sales.csv
```

Other useful argument variables:

Variable	Meaning
\$0	The script's own name
\$1, \$2, ...	The first, second, etc. argument passed in
\$#	The total number of arguments passed
\$@	All arguments, as a list

Key takeaway: *Arguments let one script run flexibly across different environments or files instead of hardcoding values inside it - directly reusing Day 3's environment-variable philosophy in a script context.*

3. Exit Codes

Every command and script returns an exit code when it finishes - a number that reports success or failure. 0 means success; any non-zero number means something went wrong.

```
python pipeline.py
echo $?          # prints the exit code of the PREVIOUS command

# 0  -> pipeline.py finished successfully
# 1  -> pipeline.py failed (most common generic failure code)
```

Using exit codes to control what happens next:

```
#!/bin/bash

python pipeline.py

if [ $? -eq 0 ]; then
    echo "Pipeline succeeded, sending success notification"
else
    echo "Pipeline failed, sending alert"
fi
```

Key takeaway: *This is exactly how tools like cron, Airflow, and CI/CD (Day 77) know whether a job succeeded or failed - they simply check the exit code. A Python script that crashes with an uncaught exception naturally returns a non-zero exit code.*

4. Shell Scripts

A shell script is simply a text file containing a sequence of bash commands, run top to bottom.

```
#!/bin/bash
# run_pipeline.sh - a complete example combining several concepts

LOG_FILE="pipeline_run.log"
DATE=$(date +%Y-%m-%d)

echo "Starting pipeline run: $DATE" >> $LOG_FILE

python pipeline.py >> $LOG_FILE 2>&1

if [ $? -eq 0 ]; then
    echo "Pipeline succeeded: $DATE" >> $LOG_FILE
else
    echo "Pipeline FAILED: $DATE" >> $LOG_FILE
fi
```

Line-by-line breakdown:

- `#!/bin/bash` - the "shebang" line, tells the system which program should run this file
- `$(date +%Y-%m-%d)` - runs the date command and stores its output in a variable
- `>> $LOG_FILE` - appends output to the log instead of overwriting it each run
- `2>&1` - redirects error messages into the same log file as normal output, so nothing gets lost

Before running a script, make it executable:

```
chmod +x run_pipeline.sh
./run_pipeline.sh
```

Key takeaway: *This one script combines variables, command substitution, exit codes, and redirection into a realistic pattern - this is genuinely close to what a production cron job looks like.*

5. cron

cron is a Linux scheduler that runs a command automatically at specified times - the simplest way to "schedule a pipeline" before you learn Airflow (Day 35).

Editing your crontab:

```
crontab -e    # opens your personal crontab file for editing
```

Crontab syntax - five time fields, then the command:

```
* * * * * command_to_run
■ ■ ■ ■ ■
■ ■ ■ ■ ■■ day of week (0-6, Sunday=0)
■ ■ ■ ■■■■ month (1-12)
■ ■ ■■■■■■ day of month (1-31)
■ ■■■■■■■■ hour (0-23)
■■■■■■■■■ minute (0-59)
```

Real examples:

```
# Run every day at 2:00 AM
0 2 * * * /home/akash/run_pipeline.sh

# Run every hour, on the hour
0 * * * * /home/akash/run_pipeline.sh

# Run every Monday at 9:00 AM
0 9 * * 1 /home/akash/run_pipeline.sh

# Run every 15 minutes
*/15 * * * * /home/akash/run_pipeline.sh
```

Key takeaway: *Always use full, absolute file paths in a crontab entry (not relative paths like ./run_pipeline.sh) - cron does not run with the same working directory or environment as your interactive terminal, and this is the single most common reason a cron job "works when I test it, but not on schedule."*

6. Running Python from Bash

Bringing it all together - a bash script that activates a virtual environment (Day 3), runs a Python pipeline, and logs the result.

```
#!/bin/bash

cd /home/akash/projects/retail-etl-pipeline
source venv/bin/activate

python pipeline.py
EXIT_CODE=$?

deactivate

if [ $EXIT_CODE -ne 0 ]; then
    echo "Pipeline failed with exit code $EXIT_CODE"
    exit 1
fi

echo "Pipeline completed successfully"
```

Why activate the virtual environment inside the script? cron and other schedulers run in a minimal environment - they don't automatically know about the virtual environment you set up interactively. The

script must activate it explicitly every time.

Key takeaway: *This pattern - cd into the project, activate venv, run the script, check the exit code - is the realistic bridge between "a Python script that works on my machine" and "a scheduled job that runs unattended on a server," and it's exactly what Day 20's Foundation Project asks you to set up.*

Day 16 Summary — Bash + Automation

Concept	One-Line Recap
Variables	No spaces around =, use \$ to read, quote paths
Arguments	\$1, \$2... let a script take input instead of hardcoding values
Exit Codes	0 = success, non-zero = failure - checked by cron, Airflow, and CI/CD
Shell Scripts	A text file of commands with a #!/bin/bash shebang, made executable with chmod +x
cron	Schedules commands via 5 time fields; always use absolute paths
Running Python from Bash	cd, activate venv, run script, check exit code - the real automation pattern

Overall takeaway: *Bash is the layer that turns a script you run manually into something that runs unattended, on schedule, and reports whether it succeeded - this is the exact gap between "I wrote a pipeline" and "I have a pipeline running in production," and it's foundational before Airflow formalizes the same ideas from Day 35 onward.*